

SPE Amplifier Remote Control

User Guide

For SPE Expert 1.5K-FA / 2K-FA HF Amplifiers

Original script by **OH2GEK**

Modernized and extended by **VU2CPL**

Version	2.0 (Python 3)
Platform	Raspberry Pi / Linux / macOS
Interface	Web browser (any device)
Connection	USB / RS-232 Serial

Table of Contents

1. Introduction
2. Requirements
3. Installation
4. Configuration
5. Starting the Server
6. Web Interface Guide
7. Controls Reference
8. Running as a System Service
9. WebSocket API
10. Troubleshooting

1. Introduction

SPE Remote Control is a modern Python 3 application that allows you to monitor and control SPE Expert HF amplifiers remotely from any web browser. It communicates with the amplifier via USB or RS-232 serial connection and serves a real-time web dashboard over your local network.

Key Features:

- **Self-contained** - single process serves both the WebSocket API and web UI. No separate web server (Apache, Nginx) needed.
- **Multi-client** - multiple browsers and devices can monitor the amplifier simultaneously.
- **Real-time gauges** - SWR, drain current, PA temperature, and voltage displayed with animated arc gauges.
- **Responsive design** - works on desktop, tablet, and mobile screens.
- **Auto-reconnect** - both WebSocket and serial connections reconnect automatically on failure.
- **Async I/O** - non-blocking serial communication using pyserial-asyncio for reliable performance.
- **Configurable** - YAML configuration file for serial port, baud rate, polling intervals, and more.

2. Requirements

Hardware:

- Raspberry Pi (any model - Pi 2, 3, 4, 5, or Zero 2 W) or any Linux/macOS computer
- SPE Expert amplifier (1.5K-FA, 2K-FA, or compatible model)
- USB cable or RS-232 serial connection to the amplifier
- Network connection (Ethernet or Wi-Fi)

Software:

- Python 3.9 or newer (included in Raspberry Pi OS)
- python3-venv package (for virtual environment)
- Git (for cloning the repository)

Python Dependencies (installed automatically):

Package	Version	Purpose
tornado	>= 6.0	Web server and WebSocket framework
pyserial-asyncio	>= 0.6	Async serial port communication
pyyaml	>= 6.0	YAML configuration file parsing

3. Installation

Step 1: Clone the Repository

```
git clone https://github.com/AnonMaoj/spe-remote.git
cd spe-remote
```

Step 2: Run Setup

```
./setup.sh
```

The setup script creates a Python virtual environment and installs all dependencies. On Raspberry Pi OS, it will also install python3-venv if needed.

Step 3: Add User to dialout Group

To access serial ports without sudo, add your user to the dialout group:

```
sudo usermod -aG dialout $USER
```

Log out and log back in for the group change to take effect.

4. Configuration

All settings are in the **config.yaml** file in the project root directory. Edit this file to match your setup.

```
serial:
  port: /dev/serial/by-id/usb-FTDI_FT232R_USB_UART_...
  baudrate: 115200
  timeout: 1.0

server:
  port: 8888
  host: "0.0.0.0"

polling:
  tx_interval: 0.2
  idle_interval: 1.0
  heartbeat: 15

logging:
  level: INFO
```

Configuration Options:

Setting	Default	Description
serial.port	/dev/ttyUSB0	Serial port path to amplifier
serial.baudrate	115200	Serial baud rate
server.port	8888	HTTP/WebSocket listen port
server.host	0.0.0.0	Listen address (0.0.0.0 = all interfaces)
polling.tx_interval	0.2	Poll interval during TX (seconds)
polling.idle_interval	1.0	Poll interval during RX/Standby
polling.heartbeat	15	Force state broadcast interval
logging.level	INFO	Log verbosity (DEBUG/INFO/WARNING/ERROR)

Finding Your Serial Port:

```
# List USB serial devices
ls /dev/serial/by-id/

# Or check dmesg after plugging in the USB cable
dmesg | grep ttyUSB
```

Tip: Use /dev/serial/by-id/... paths instead of /dev/ttyUSB0. The by-id paths are stable across reboots and don't change if other USB devices are connected.

5. Starting the Server

Start with the run script:

```
./run.sh
```

Or start directly with the virtual environment Python:

```
venv/bin/python server.py
```

To use a custom config file:

```
venv/bin/python server.py /path/to/custom-config.yaml
```

On successful startup you will see:

```
2024-01-15 10:30:00 [INFO] spe: Server listening on http://0.0.0.0:8888/  
2024-01-15 10:30:00 [INFO] spe: Serial port: /dev/ttyUSB0 @ 115200 baud  
2024-01-15 10:30:00 [INFO] spe.serial_handler: Connecting to /dev/ttyUSB0...  
2024-01-15 10:30:00 [INFO] spe.serial_handler: Serial connected
```

Open your browser and navigate to **`http://<pi-ip-address>:8888/`**

6. Web Interface Guide

The web interface provides a real-time view of the amplifier's status with interactive controls.

Connection Indicator

The green/red dot at the top shows WebSocket connection status. If disconnected, the client will automatically attempt to reconnect with increasing delays.

Power Output Display

The large power bar shows the current output power in watts (0-1500W). The gradient color changes from green through cyan and yellow to red as power increases. SWR is displayed in the top-right corner.

Gauges

Four semi-circular arc gauges display real-time readings:

Gauge	Range	Warning Zone
SWR	1:1 to 1:3.5	Above 1:2.0 (orange), above 1:2.8 (red)
Drain Current	0 - 60 A	Above 42 A (orange), above 51 A (red)
PA Temperature	0 - 80 C	Above 60 C (orange), above 64 C (red)
Voltage	40 - 60 V	Below 44 V or above 55 V (warning)

Status Chips

A row of status chips shows: TX/RX status (red pulse during TX, green during RX), current band, antenna number, input number, and power level setting.

Alert Bar

Warning and error messages from the amplifier are displayed at the bottom in an orange (warning) or red (error) bar.

7. Controls Reference

Button	Action	Description
Operate	Toggle Operate/Standby	Switches the amplifier between Operate and Standby modes. Button highlights c
ANT	Cycle TX Antenna	Cycles through available TX antenna outputs (ANT 1, ANT 2, etc.).
TUNE	Start ATU Tuning	Initiates the internal antenna tuning unit (ATU) cycle. Ensure RF drive is present.
INPUT	Toggle Input	Switches between available input connectors.
GAIN	Toggle Gain	Cycles through gain/power level settings.

Note: All connected clients can send commands. In a multi-client setup, coordinate with other operators to avoid conflicting commands.

8. Running as a System Service

To start the server automatically on boot, create a systemd service file:

Step 1: Create the Service File

```
sudo nano /etc/systemd/system/spe-remote.service
```

Paste the following content:

```
[Unit]
Description=SPE Amplifier Remote Control
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/spe-remote
ExecStart=/home/pi/spe-remote/venv/bin/python server.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Adjust User and WorkingDirectory to match your actual username and install path.

Step 2: Enable and Start

```
sudo systemctl daemon-reload
sudo systemctl enable spe-remote
sudo systemctl start spe-remote
```

Step 3: Check Status

```
sudo systemctl status spe-remote
journalctl -u spe-remote -f
```

9. WebSocket API

The server exposes a WebSocket endpoint for custom clients and integrations.

Endpoint:

ws://<host>:8888/ws

Received JSON (amplifier state):

```
{
  "op_status": "Oper",
  "tx_status": "TX",
  "band": "80m",
  "p_out": "1353",
  "swr": "1.54",
  "voltage": "54.6",
  "drain": "27.3",
  "pa_temp": "26",
  "warnings": "",
  "error": ""
}
```

Send Commands (text messages):

Command	Action
oper	Toggle Operate/Standby
antenna	Cycle TX antenna
input	Toggle input
tune	Start ATU tuning
gain	Toggle gain

10. Troubleshooting

Problem: Serial error: No such file or directory

Solution: The serial port path in config.yaml does not exist. Check the cable is plugged in and verify the port with: `ls /dev/serial/by-id/`

Problem: Permission denied on serial port

Solution: Add your user to the dialout group: `sudo usermod -aG dialout $USER`, then log out and back in.

Problem: Web page not loading

Solution: Check that the server is running and the port is not blocked by a firewall. Try: `sudo ufw allow 8888/tcp`

Problem: Gauges not updating

Solution: Open browser developer console (F12) and check for WebSocket connection errors. Verify the server IP and port are correct.

Problem: Multiple /dev/ttyUSB devices

Solution: Use the `/dev/serial/by-id/` path which is unique to each USB-serial adapter. This prevents confusion when multiple USB devices are connected.

Problem: Server crashes on startup

Solution: Check the log output for errors. Common causes: wrong Python version (need 3.9+), missing dependencies (re-run `./setup.sh`), or serial port already in use by another program.

Problem: High CPU usage

Solution: Increase `polling.idle_interval` in config.yaml to reduce serial polling frequency when idle. The default 1.0 second should be fine for most setups.

For additional help, open an issue on the GitHub repository.